Refactor Pipeline 3PL

1. Objetivo

Se realizó un refactor del pipeline de **3PLICITY Frontend** para separar la configuración y ejecución de las pruebas automatizadas de Cypress de la definición principal del pipeline, permitiendo reutilizar la misma configuración en diferentes escenarios.

El objetivo principal fue mantener el flujo normal de entrega de la aplicación **(CD)**:

```
Build → Cypress → Deploy
```

y, al mismo tiempo, disponer de un pipeline independiente que permita ejecutar pruebas de regresión de Cypress manualmente contra una aplicación que ya se encuentra desplegada.

La nueva implementación utiliza tres archivos YAML:

```
azure-pipelines.yml
cypress-pipeline.yml

templates/
└── cypress-tests.yml
```

El archivo `cypress-tests.yml` contiene la lógica común de Cypress y es reutilizado por los dos pipelines.

---

2. Flujo anterior

Antes del refactor, la configuración de Cypress se encontraba directamente dentro de `azure-pipelines.yml`.

El pipeline tenía el siguiente flujo:

```
PR Merge
    │
    ▼
 BUILD
    │
    ├── dev  → npm run build:dev
    │
    └── main → npm run build:prod
    │
    ▼
Publish dist
    │
```

```
    ▼
CYPRESS
    │
    ├── dev  → Cypress con configuración DEV
    │
    └── main → Cypress con configuración PROD
    │
    ▼
DEPLOY
    │
    ├── dev  → Azure Static Web App DEV
    │
    └── main → Azure Static Web App PROD
```

En este esquema, Cypress dependía directamente del resultado del Build.

Después de generar el `dist`, el pipeline lo descargaba en el job de Cypress, levantaba la aplicación localmente utilizando `serve` y esperaba a que estuviera disponible mediante `wait-on`.

La configuración de Cypress también incluía directamente:

- Instalación de Node.js.
- Instalación de dependencias mediante `npm ci`.
- Instalación de `serve` y `wait-on`.
- Configuración de `CYPRESS_BASE_URL`.
- Variables de API y credenciales de prueba.
- Ejecución de `npx cypress run`.
- Publicación de reportes.
- Publicación de screenshots.
- Publicación de resultados JUnit mediante `PublishTestResults@2`.

---

3. Problema identificado

El principal inconveniente de mantener toda esta configuración dentro de `azure-pipelines.yml` era que la ejecución de Cypress estaba completamente acoplada al pipeline de entrega.

Esto impedía utilizar la misma configuración de Cypress para un escenario independiente de regresión que estuviera a la mano para el equipo completo y no solamente para QA.

Además, el pipeline manual no necesita generar un nuevo `dist`, ya que su propósito es ejecutar pruebas contra una versión de la aplicación que ya se encuentra desplegada en un ambiente.

Por lo tanto, se identificaron dos escenarios diferentes:

Pipeline de entrega

```
Build
  ↓
```

```
dist
  ↓
serve localhost:8080
  ↓
Cypress
  ↓
Deploy
```

Pipeline manual de regresión

```
Aplicación ya desplegada
  ↓
Selector DEV / PROD
  ↓
Cypress
  ↓
Reports
```

La solución fue extraer la lógica común de Cypress a un template reutilizable.

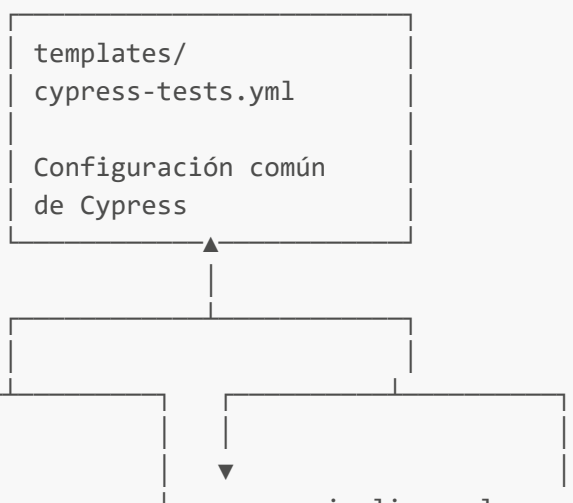
---

4. Arquitectura implementada

La arquitectura final utiliza tres archivos:

```
azure-pipelines.yml
cypress-pipeline.yml
templates/
└── cypress-tests.yml
```

La relación entre ellos es:



```
                                  |
        Build                     | Ejecución manual
          ↓                       |
        Cypress ──────────────────┘        ↓
          ↓                              Cypress
        Deploy                             ↓
                                         Reports
```

De esta manera, existe una sola implementación de la lógica de Cypress, pero puede ejecutarse en diferentes contextos.

---

5. `templates/cypress-tests.yml`

Este archivo contiene la configuración reutilizable de Cypress.

Su objetivo es encapsular las tareas necesarias para preparar el ambiente, ejecutar las pruebas y publicar los resultados.

La configuración contempla dos escenarios mediante parámetros:

```yaml
parameters:
  - name: useLocalBuild
    type: boolean
    default: true

  - name: baseUrl
    type: string
    default: "http://localhost:8080"

  - name: environment
    type: string
    default: "dev"
```

5.1 Parámetro `useLocalBuild`

Determina si Cypress debe utilizar el `dist` generado por el Build.

`true`

Se utiliza en el pipeline normal de entrega.

```
Build
 ↓
vite-dist
 ↓
Download artifact
 ↓
```

```
serve localhost:8080
 ↓
Cypress
```

`false`

Se utiliza en el pipeline manual.

En este caso no se descarga ni se levanta un `dist` local, porque Cypress apunta directamente a una aplicación ya desplegada.

```
DEV / PROD desplegado
 ↓
Cypress
```

5.2 Parámetro `baseUrl`

Permite definir la aplicación contra la cual se ejecutarán las pruebas.

En el pipeline normal:

```
baseUrl: "http://localhost:8080"
```

En el pipeline manual:

```
baseUrl: "https://delightful-ground-005697610.5.azurestaticapps.net"
```

para DEV, o:

```
baseUrl: "https://app.3plicity.com"
```

para PROD.

Esto permite mantener una única configuración de Cypress sin tener que duplicar el código.

5.3 Parámetro `environment`

Determina qué configuración de ambiente debe utilizar Cypress:

```
environment: "dev"
```

o:

```
environment: "prod"
```

El template utiliza este parámetro para seleccionar la ejecución correspondiente y las variables de Cypress del ambiente seleccionado.

---

6. Funcionamiento del template

Cuando `useLocalBuild` es `true`, el template realiza las siguientes acciones:

```
Install Node.js
      ↓
Download vite-dist
      ↓
Install serve + wait-on
      ↓
Start application on localhost:8080
      ↓
Wait for application
      ↓
Install Cypress dependencies
      ↓
Run Cypress
      ↓
Publish reports
      ↓
Publish screenshots
      ↓
Publish JUnit results
```

Cuando `useLocalBuild` es `false`, se omiten las tareas relacionadas con el Build local:

```
Install Node.js
      ↓
Install Cypress dependencies
      ↓
Run Cypress against already deployed URL
      ↓
Publish reports
      ↓
```

```
Publish screenshots
      ↓
Publish JUnit results
```

---

7. azure-pipelines.yml

El pipeline principal continúa siendo responsable del flujo de entrega de la aplicación.

Su arquitectura permanece:

```
BUILD
  ↓
CYPRESS
  ↓
DEPLOY
```

La diferencia es que la configuración detallada de Cypress ya no se encuentra directamente dentro del archivo.

En su lugar, el pipeline referencia:

```
- template: templates/cypress-tests.yml
```

7.1 Build

El Build continúa generando el artefacto `vite-dist`.

Para `dev`:

```
npm run build:dev
```

Para `main`:

```
npm run build:prod
```

Posteriormente el contenido de `dist` se publica como:

```
vite-dist
```

Este artefacto es utilizado posteriormente por Cypress y por los stages de Deploy.

7.2 Cypress

El stage de Cypress utiliza el template con:

```
useLocalBuild: true
```

porque necesita probar exactamente el build generado por el stage anterior.

Para DEV:

```
- ${{ if eq(variables['Build.SourceBranch'], 'refs/heads/dev') }}:
    - template: templates/cypress-tests.yml
      parameters:
        useLocalBuild: true
        baseUrl: "http://localhost:8080"
        environment: "dev"
```

Para PROD:

```
- ${{ if eq(variables['Build.SourceBranch'], 'refs/heads/main') }}:
    - template: templates/cypress-tests.yml
      parameters:
        useLocalBuild: true
        baseUrl: "http://localhost:8080"
        environment: "prod"
```

La inclusión del template se realiza mediante una condición de compilación de Azure DevOps.

Esto permite que solamente se incluya la configuración correspondiente a la rama que está ejecutando el pipeline.

---

8. cypress-pipeline.yml

Se creó un segundo pipeline independiente exclusivamente para ejecutar las pruebas automatizadas.

Este pipeline no realiza:

```
Build
Deploy
```

Su flujo es:

```
Run pipeline
      ↓
Seleccionar ambiente
      ↓
DEV / PROD
      ↓
Cypress
      ↓
Reports
```

El pipeline se configuró con:

```
trigger: none
pr: none
```

Por lo tanto, su ejecución es manual.

8.1 Selección de ambiente

El pipeline recibe un parámetro:

```
parameters:
  - name: environment
    displayName: "Environment"
    type: string
    default: "dev"
    values:
      - dev
      - prod
```

Al ejecutar el pipeline, se puede seleccionar:

```
Environment:
[ DEV ]
```

o:

```
Environment:
[ PROD ]
```

8.2 Ejecución contra DEV

Cuando se selecciona `dev`, el pipeline utiliza:

```
useLocalBuild: false
baseUrl: "https://delightful-ground-005697610.5.azurestaticapps.net"
environment: "dev"
```

Por lo tanto, Cypress ejecuta las pruebas directamente contra la aplicación desplegada en DEV.

8.3 Ejecución contra PROD

Cuando se selecciona `prod`, el pipeline utiliza:

```
useLocalBuild: false
baseUrl: "https://app.3plicity.com"
environment: "prod"
```

En este caso, Cypress ejecuta las pruebas directamente contra la aplicación desplegada en PROD.

---

9. Variables de Cypress

La configuración continúa utilizando las variables existentes de Azure DevOps para cada ambiente.

Para DEV se utilizan variables como:

```
CYPRESS_API_URL
CYPRESS_3PL
CYPRESS_MERCHANT
CYPRESS_CARD
CYPRESS_EXPIRE
CYPRESS_CVC
CYPRESS_CUPON
```

Para PROD:

```
CYPRESS_API_URL_PROD
CYPRESS_3PL_PROD
CYPRESS_MERCHANT_PROD
CYPRESS_CARD_PROD
CYPRESS_EXPIRE_PROD
```

```
CYPRESS_CVC_PROD
CYPRESS_CUPON_PROD
```

La URL base no necesita mantenerse como una variable independiente para el pipeline manual, ya que se proporciona mediante el parámetro `baseUrl`.

---

10. Reportes y resultados

La generación y publicación de resultados permanece dentro del template para que ambos pipelines utilicen el mismo mecanismo.

Se publican:

Report

Contiene los reportes generados por Cypress.

```
Report
```

Screenshots

Contiene las capturas generadas durante las pruebas.

```
Screenshots
```

JUnit

Los resultados JUnit se publican mediante:

```
PublishTestResults@2
```

Para DEV:

```
Cypress E2E (dev)
```

Para PROD:

```
Cypress E2E (prod)
```

Esto permite visualizar los resultados desde la sección **Tests** de Azure DevOps.

---

11. Consideración sobre Pull Requests

Durante la implementación se identificó un comportamiento importante relacionado con las condiciones de Azure DevOps.

La ejecución de validación de un Pull Request utiliza una referencia temporal similar a:

```
refs/pull/<id>/merge
```

y no:

```
refs/heads/dev
```

o:

```
refs/heads/main
```

Debido a que la inclusión del template de Cypress está condicionada por la rama:

```
${{ if eq(variables['Build.SourceBranch'], 'refs/heads/dev') }}
```

o:

```
${{ if eq(variables['Build.SourceBranch'], 'refs/heads/main') }}
```

los pasos de Cypress no se incluyen durante esa validación cuando Azure está ejecutando el merge temporal del PR.

Esto se observó durante la validación del cambio:

```
Build
  ↓
Cypress Job
  ↓
Cypress steps omitidos
```

```
  ↓
Deploy omitido
```

Después de realizar el merge a `dev`, el pipeline se ejecutó nuevamente sobre la rama correspondiente y Azure pudo evaluar correctamente:

```
refs/heads/dev
```

En esa ejecución se confirmó el comportamiento esperado:

```
Build
  ↓
Download artifact
  ↓
Install serve + wait-on
  ↓
Start application
  ↓
Wait for application
  ↓
Install Cypress dependencies
  ↓
Run Cypress Tests Dev
  ↓
Reports / Screenshots / JUnit
  ↓
Deploy Dev
```

Este comportamiento fue validado posteriormente con una ejecución exitosa del pipeline.

---

12. Consideración sobre `condition` y templates

Durante la implementación se presentó un conflicto al intentar colocar directamente una propiedad `condition` sobre una referencia de template:

```
- template: templates/cypress-tests.yml
  condition: ...
```

Azure DevOps no permite utilizar `condition` de esta forma sobre el elemento `template`.

La solución implementada fue utilizar una condición de compilación para incluir el template:

```
- ${{ if eq(variables['Build.SourceBranch'], 'refs/heads/dev') }}:
    - template: templates/cypress-tests.yml
```

y de forma equivalente para PROD.

Esta estructura permite que Azure DevOps determine durante la expansión del YAML si debe incluir el template correspondiente.

---

13. Validación realizada

Antes de realizar los cambios se creó un **backup maestro de `azure-pipelines.yml`**.

Posteriormente se realizaron las siguientes actividades:

1. Identificación de las tareas correspondientes a Build, Cypress y Deploy.
2. Extracción de la configuración reutilizable de Cypress.
3. Creación de `templates/cypress-tests.yml`.
4. Modificación de `azure-pipelines.yml` para utilizar el template.
5. Creación de `cypress-pipeline.yml`.
6. Configuración del pipeline manual para DEV y PROD.
7. Validación de la ejecución manual contra DEV.
8. Validación de la ejecución manual contra PROD.
9. Creación de Pull Request para validar los cambios.
10. Corrección de la inclusión condicional del template.
11. Merge de los cambios a `dev`.
12. Validación del pipeline normal después del merge.

Los escenarios principales fueron ejecutados correctamente.

---

14. Arquitectura final

La implementación final permite utilizar Cypress en dos contextos:

Pipeline de entrega

```
azure-pipelines.yml

Build
  ↓
Generate dist
  ↓
Cypress
  ↓
Reports
```

```
  ↓
Deploy
```

Cypress prueba el build generado por el mismo pipeline utilizando:

```
localhost:8080
```

Pipeline manual de regresión

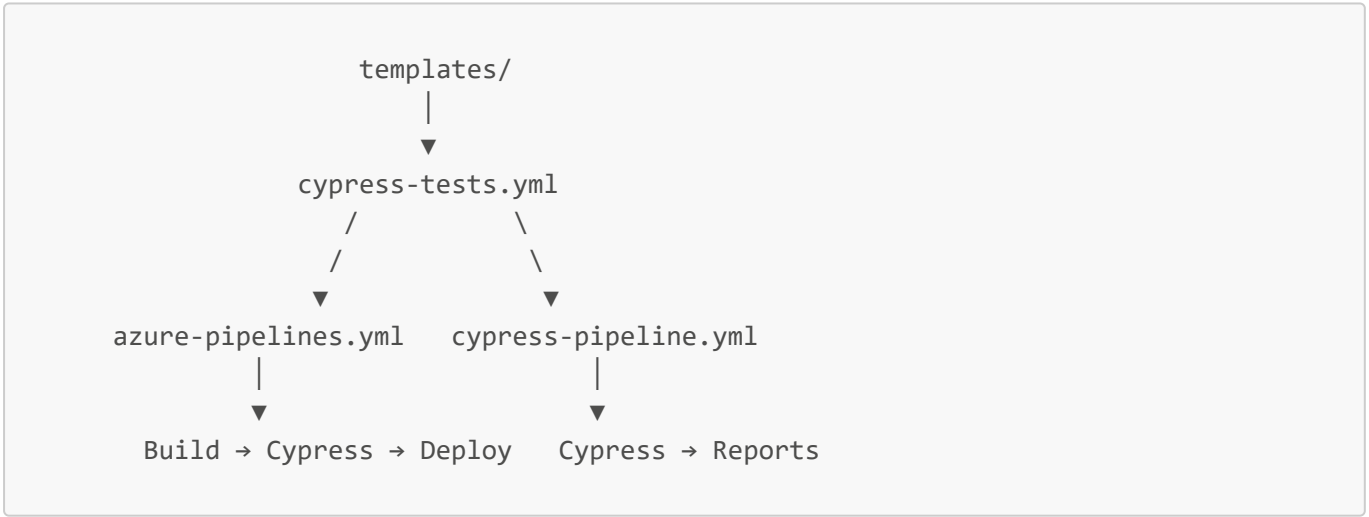
Nombre del Pipeline: 3PL Cypress Manual Regression

```
cypress-pipeline.yml

Seleccionar DEV / PROD
  ↓
Cypress
  ↓
Reports
```

Cypress prueba directamente una aplicación ya desplegada utilizando la URL correspondiente al ambiente.

Configuración compartida



---

15. Resultado

Con este refactor, la configuración de Cypress dejó de estar acoplada directamente a un único pipeline y pasó a ser una **unidad reutilizable mediante un template de Azure DevOps**.

La solución permite:

- Mantener el pipeline de entrega existente.

- Ejecutar Cypress sobre el build recién generado.
- Ejecutar regresiones manualmente sin realizar un nuevo Build.
- Seleccionar DEV o PROD desde el pipeline manual.
- Reutilizar la misma lógica de Cypress en ambos escenarios.
- Mantener la generación de reportes, screenshots y resultados JUnit.
- Evitar duplicación de la configuración de Cypress.
- Mantener separados los casos de uso de **delivery** y **regresión manual**.

La arquitectura resultante permite que futuras modificaciones a la configuración común de Cypress se realicen principalmente en:

```
templates/cypress-tests.yml
```

en lugar de mantener configuraciones duplicadas en múltiples pipelines.
